Labor: Place Recognition (Monte Carlo Labyrinth)

Fabian Alexander Feith
Inalbek Akiev
Fakultät 10 | Robotik | Prof. Dr. Algorri

Ziel dieses Labors ist die Lösung des Kidnapped-Robot-Problems im Monte-Carlo-Labyrinth: Der Roboter wird an einem von vier vorher gelernten Orten (P2, P4, P6, P9) in beliebiger Orientierung ausgesetzt, erkennt per Rundum-Scan, wo er steht, bestimmt seine Verdrehung gegenüber der Trainingsausrichtung, richtet sich neu aus und fährt zu den nächsten Wegpunkten weiter. Die Umsetzung baut bewusst auf dem bestehenden mcl_nav-Paket aus dem Monte-Carlo-Projekt auf, verändert es aber nicht: Das Wandmodell aus map_config.yaml und die Raycast-Funktion aus mcl.py werden nur lesend wiederverwendet, ebenso das Teensy-Kommandoprotokoll (T:/R:/S: mit Ist-Feedback) für die Fahrphase.

Der Code ist zweigeteilt: Ein Diagnose- und Entwicklungsordner place_recognition_dev enthält die Kernlogik (place_signature.py), die Lern- und Erkennungsprogramme (learn_location.py, recognize_location.py), die Visualisierung (visualize_signature.py, compare_resolutions.py) und die Offline-Kalibrierung (synthetic_scan.py, validate_offline.py). Die eigentliche Fahrstufe ist eine eigenständige ROS2-Node pr_navigator.py im mcl_nav-Paket. Die Diagnoseprogramme senden grundsätzlich keine Fahrkommandos, und ein eigenes Launchfile (lidar_only.launch.py) startet nur den Lidar, damit der Roboter beim manuellen Positionieren während des Trainings nicht losfährt.

Rundum-Scan und Signatur

Da der RPLidar A1 ohnehin volle 360 Grad abdeckt, muss sich der Roboter für den Rundum-Scan nicht drehen (die Sonar-Variante der Aufgabenstellung entfällt). Eine Ortssignatur ist ein Array aus NO_BINS = 360 Tiefenwerten über den Vollkreis, Bin 0 zeigt in Blickrichtung des Roboters bei der Aufnahme. Die Zuordnung der 360 bis 460 Rohmesspunkte pro Umdrehung zu den Winkel-Bins ist mit numpy vektorisiert (Mittelwert aller Punkte, die in einen Bin fallen), Bins ohne Wandtreffer bleiben unendlich:

```
bin_idx = np.round(angles_norm / self._bin_width).astype(int) % self.n_bins
sums   = np.bincount(bin_idx[finite], weights=ranges[finite], minlength=self.n_bins)
counts = np.bincount(bin_idx[finite], minlength=self.n_bins)
binned[has_data] = sums[has_data] / counts[has_data]
```

Gegen Messrauschen wird nicht ein einzelner Scan verwendet, sondern der Mittelwert aus n_scans = 10 aufeinanderfolgenden Scans (bin-weise, inf-Werte ausgenommen). Beim Training wird der Roboter an jedem der vier Orte von Hand exakt in die dortige lokale Fahrtrichtung gestellt; wichtig ist, dass das nicht eine globale Richtung für alle Orte ist, denn die Route wechselt im Labyrinth mehrfach die Richtung (P2 und P9 zeigen physisch nach Westen, P4 und P6 nach Osten). Die Signatur wird mit dem Ortsnamen als Pickle-Datei gespeichert. Zur Kontrolle stellt visualize_signature.py jede Signatur als radiale Linien um den Robotermittelpunkt dar (auch live ohne Speichern), und compare_resolutions.py zeichnet dieselbe Signatur bei 36, 72, 180 und 360 Bins nebeneinander. An echten P2-

Daten zeigt sich: Die charakteristische Kontur ist schon bei 36 Bins erkennbar, 360 Bins entsprechen aber etwa der nativen Auflösung des Lidars, also bleibt es bei 360.

Offline-Kalibrierung gegen das Wandmodell

Bevor der Roboter überhaupt fuhr, wurde die komplette Erkennungskette offline kalibriert. synthetic_scan.py erzeugt aus dem Wandmodell der Karte per Raycast künstliche 360-Grad-Scans an beliebigen Posen, validate_offline.py simuliert damit 200 verrauschte Testaufstellungen pro Ort (Gauß-Rauschen mit dem lidar_sigma = 0,01 m aus der MCL-Konfiguration, zufällige Orientierung) und testet mehrere Bin-Anzahlen:

NO_BINS	Genauigkeit	Dk korrekt(max)	Dk falsch(min)	Winkelfehler mean/max
36	100.0%	68	42	3.81/9.92 deg
72	100.0%	93	160	1.68/4.94 deg
180	100.0%	203	919	0.69/2.00 deg
360	100.0%	589	3918	0.34/1.00 deg

Alle vier Orte sind im Wandmodell strukturell klar unterscheidbar. Als Schwellenwert für ein gültiges Match wurde die Mitte zwischen dem größten korrekten und dem kleinsten falschen Dk-Wert bei 360 Bins gewählt: Dk_threshold = 2253. Bewusst festgehalten wurde auch eine Einschränkung dieser Auswertung: Die Dk-Werte sind nicht über die Bin-Anzahl normiert, der scheinbare Margengewinn bei mehr Bins ist also teilweise ein Rechenartefakt. Der Schwellenwert wurde deshalb später an echten Hardwaredaten überprüft, bevor er endgültig übernommen wurde.

Stufe 1: Erkennung bei fester Orientierung

Der Vergleichstest ist die Summe der quadratischen Abweichungen zwischen Test- und Trainingssignatur. Steht der Roboter beim Test wie beim Training in Fahrtrichtung, genügt dieser direkte Vergleich der Winkel-Signaturen. Der Ort mit dem kleinsten Dk unterhalb des Schwellenwerts gewinnt, sonst wird kein Match gemeldet:

```
def dk(hist_a, hist_b):
    return float(np.sum((np.asarray(hist_a) - np.asarray(hist_b)) ** 2))
```

Im ersten Hardwaretest wurden alle vier Orte korrekt erkannt (4/4), mit deutlicher Marge: Die Dk-Werte des jeweils richtigen Ortes lagen zwischen 103 und 1393, die aller falschen Orte zwischen 5211 und 19548. Der offline kalibrierte Schwellenwert 2253 trennt beide Bereiche sauber und wurde damit an echten Daten bestätigt.

Stufe 2: Rotationsinvariante Erkennung und Winkelschätzung

Steht der Roboter beliebig verdreht, verschiebt sich die Winkel-Signatur zirkulär und der direkte Vergleich versagt. Unser Ansatz sah demnach wiefolgt aus. Erstens ein

rotationsinvariantes Tiefen-Frequenz-Histogramm: Statt Tiefe über Winkel wird gezählt, wie oft jede Tiefe vorkommt (50 Tiefen-Bins zu je 10 cm zwischen 0 und 5 m). Dieses Histogramm ändert sich bei Drehung nicht und erlaubt einen schnellen Ortsvergleich. Zweitens die Winkelschätzung: Für den Kandidaten wird die gespeicherte Winkel-Signatur über alle 360 möglichen Bin-Verschiebungen zirkulär gegen die Test-Signatur korreliert; die Verschiebung mit dem kleinsten Dk ist die gesuchte Verdrehung:

```
for shift in range(n):
    shifted = np.roll(stored_v, shift)
    dks[shift] = np.sum((test_v - shifted) ** 2)
best_shift = int(np.argmin(dks))   # -> Winkel = shift * 360 / n_bins
```

Die Hardwaretests deckten zwei Schwächen der ersten Version auf, beide sind im Changelog dokumentiert und behoben. Erstens verwechselte das Frequenz-Histogramm bei um 90 Grad gedrehtem Roboter die ähnlichsten Orte P2 und P4. Die Erkennung wurde deshalb zweistufig: Das Histogramm liefert nur noch die zwei wahrscheinlichsten Kandidaten als Vorfilter, und unter diesen entscheidet der genauere Verschiebungs-Vergleich der vollen Winkel-Signaturen. Zweitens konnte ein Kandidat, der laut Histogramm klar über dem Schwellenwert lag (P9 mit Dk = 10729), durch einen zufällig günstigen Verschiebungs-Dk den echten Treffer P6 verdrängen und ein KEIN MATCH erzeugen. Seitdem nimmt der Feinvergleich nur Kandidaten an, die unterhalb des Schwellenwerts liegen. Zusätzlich wurde die anfangs falsch herum interpretierte Vorzeichenkonvention der Winkelschätzung an der Hardware verifiziert: positiv bedeutet, der Roboter steht im Uhrzeigersinn rechts von der Trainingsausrichtung, die Rückdrehung ist damit T:+winkel (inklusive des empirischen Teensy-Skalierungsfaktors 1,03).

Nach diesen Korrekturen und einem Neu-Lernen der empfindlichen P2-Signatur wurde systematisch getestet: alle vier Orte, jeweils bei 0 Grad, 90 Grad rechts, 180 Grad und 90 Grad links, insgesamt 16 Versuche. Ergebnis: alle Orte in allen Orientierungen korrekt erkannt, Winkelschätzung durchgehend konsistent (z. B. P9 bei 90 Grad links: geschätzte Abweichung -89,0 Grad). Lehrreich war ein Ausreißer bei P9, der sich als Messstörung durch einen fremden Roboter im Sichtfeld bei P2 herausstellte. Nach dem Entfernen des Hindernisses fiel der Dk-Wert von 765 auf 77 und das Vorzeichen stimmte. Die Methode reagiert also erwartungsgemäß empfindlich auf dynamische Objekte, erkannte den Ort in diesem gestörten Fall aber trotzdem korrekt.

Stufe 3: Reorientierung und Weiterfahrt (pr_navigator.py)

Die dritte Bewertungsstufe übernimmt eine eigenständige Node pr_navigator.py. Sie erkennt den Startort, richtet den Roboter neu aus und fährt die jeweils nächsten zwei Wegpunkte der Monte-Carlo-Route an. Nach der Erkennung wird das aktuelle Heading rekonstruiert: Jeder

Trainingsort hat sein bekanntes Trainings-Heading (P2, P9 nach Westen; P4, P6 nach Osten), und die per Kreuzkorrelation geschätzte Abweichung wird davon abgezogen. Die Differenz zum Ziel-Heading des nächsten Wegpunkts ergibt die Reorientierungs-Drehung:

```
training_h  = {'P2': east, 'P4': west, 'P6': west, 'P9': east}[loc]
corrected_h = math.atan2(math.sin(training_h - angle_rad),
                         math.cos(training_h - angle_rad))
target_h    = {'P2': south, 'P4': north, 'P6': south, 'P9': north}[loc]
dtheta      = math.degrees(self._angle_diff(target_h, corrected_h))
```

Danach fährt die Node in kurzen Segmenten von 8 cm und stoppt per Lidar-Frontstopp auf die aus dem Monte-Carlo-Projekt bekannten Wandabstände (z. B. 0,68 m auf dem Weg von P9 zu P10), dreht am ersten Wegpunkt um den festen Routenwinkel und wiederholt das Spiel bis zum zweiten Wegpunkt. Für den Start bei P2 wird die komplette Westfahrt-Logik aus dem Hauptnavigator übernommen, inklusive der Drehwinkelkorrektur über den gemessenen Winkel der Vorderwand und der Wandfolge-Korrektur entlang der unteren Wand (Sollabstand 0,36 m, Verstärkung 30 Grad pro Meter Fehler, begrenzt auf 15 Grad). Ein empirischer Befund aus den Fahrtests ist ein konstanter Offset von +3 Grad bei der Süddrehung an P2, der die dort beobachtete Winkelunterschätzung ausgleicht. Auf einen Partikelfilter verzichtet diese Node bewusst. Nach erfolgreicher Ortserkennung ist die Position bekannt, und für zwei Wegpunkte sind die direkten Lidar-Wandreferenzen die einfachere und robustere Lösung. Nach der großen Reorientierungs-Drehung wird auch auf IMU-Korrekturen verzichtet, da der BNO055 danach unkalibriert ist. Die Geradeausfahrt stimmt, weil die Initialausrichtung aus der Kreuzkorrelation kommt und der Teensy die Radwege selbst abgleicht.

Arbeitsweise und Datendisziplin

Zwei organisatorische Entscheidungen haben sich bewährt. Erstens die strikte Trennung von echten und synthetischen Daten. Nachdem ein Kalibrierlauf die synthetischen Platzhalter-Signaturen im Betriebsordner mit echten Daten kollidieren ließ, schreibt validate_offline.py ausschließlich in einen eigenen Ordner signatures_synthetic, und signatures enthält nur noch an der Hardware gelernte Signaturen. Zweitens wurde jede Änderung mit Testergebnis im Changelog festgehalten, inklusive der Fehlversuche. Der Versuch, den P2/P4-Fehlfall synthetisch zu reproduzieren, scheiterte und ist als offener Punkt dokumentiert statt stillschweigend verworfen.

Gesamtsystem

Insgesamt entsteht eine vollständige Kette für globale Lokalisierung: Rundum-Signatur aufnehmen, Ort über das rotationsinvariante Histogramm vorfiltern, unter den Kandidaten per

zirkulärer Kreuzkorrelation Ort und Verdrehung bestimmen, reorientieren und entlang der bekannten Route weiterfahren. Die wichtigsten Stellschrauben sind die Bin-Anzahl (360, entsprechend der Lidar-Auflösung), der offline kalibrierte und an Hardware bestätigte Schwellenwert Dk = 2253, die Mittelung über 10 Scans, die Kandidatenzahl 2 des Vorfilters und die Tiefen-Bin-Breite von 10 cm des Histogramms. Stufe 1 und Stufe 2 wurden an der Hardware vollständig validiert (alle vier Orte, alle vier Orientierungen korrekt), Stufe 3 ist implementiert und im Labyrinth erprobt.